

拓扑 AC 2025 线上 CSP-S & NOIP 联测专题模拟赛 第七场讲评

王天竹

September 5, 2026

逆向冒泡

给定一个经过 k 次冒泡排序后的排列，求字典序最小的原序列

逆向冒泡

逆向冒泡

子任务一：爆搜所有的排列，然后跑 k 轮冒泡

核心观察

核心观察

对于一个数 x ，如果前面有比他大的数，那么一次冒泡之后前进一次。

核心观察

对于一个数 x ，如果前面有比他大的数，那么一次冒泡之后前进一次。
对于 k 轮冒泡之后，如果一个数 a_i 前面仍然有比他大的数，则他原来的位置必然是 $i + k$

逆向冒泡

逆向冒泡

接下来考虑，如何放置剩下的数；

逆向冒泡

接下来考虑，如何放置剩下的数；剩下的数，均为冒泡后的前缀最大值；

逆向冒泡

接下来考虑，如何放置剩下的数；剩下的数，均为冒泡后的前缀最大值；由于保证一定有解，所以从前往后考虑贪心的放这些数就可以了；

非负前缀

给定一个数组 a ，找出字典序最小的排列使得重排后的数组能够满足前缀和非负的限制；

非负前缀

非负前缀

子任务一：枚举所有的排列

非负前缀

子任务一：枚举所有的排列

子任务二：一个相对比较容易想到的贪心算法，每次从可以选的范围里选出一个下标最小的数，直接模拟这个过程，复杂度为 $O(n^2)$ ；

非负前缀

非负前缀

考虑加速这个贪心的过程，记当前选的数必须大于 x ；

非负前缀

考虑加速这个贪心的过程，记当前选的数必须大于 x ；

考虑对数组建立一颗线段树来维护 $\max$ ，每次在线段树上找到第一个大于等于 x 的位置；

非负前缀

考虑加速这个贪心的过程，记当前选的数必须大于 x ；

考虑对数组建立一颗线段树来维护 $\max$ ，每次在线段树上找到第一个大于等于 x 的位置；

选出这个数后，把这个位置改为 $-\infty$ ；

非负前缀

考虑加速这个贪心的过程，记当前选的数必须大于 x ；

考虑对数组建立一颗线段树来维护 $\max$ ，每次在线段树上找到第一个大于等于 x 的位置；

选出这个数后，把这个位置改为 $-\infty$ ；

时间复杂度为 $O(n \log n)$ 的复杂度；

树链剖分

给定一颗树，求以每个点为根的时候，树链剖分的代价最小值（代价为所有路径中轻边数量和）；

树链剖分

树链剖分

子任务一：枚举每个顶点，搜索所有的划分链的方案，然后计算答案

树链剖分

子任务一：枚举每个顶点，搜索所有的划分链的方案，然后计算答案
子任务二：最多只有 1 条轻边，枚举哪条边成为轻边作为答案即可

树链剖分

树链剖分

一个相对简单或者你学过树链剖分就应该知道的性质：每个点的出边里只会有 1 条重边；

树链剖分

一个相对简单或者你学过树链剖分就应该知道的性质：每个点的出边里只会有 1 条重边；

子任务三：枚举每个点作为根。当根固定时，由于每个点只会有一条重边，所以我们显然是选择能贡献权值最大的边作为重边。（贡献权值为该边两端连通块乘积）。我们就可以做到单次线性的解决这个问题；

树链剖分

实际上我们发现，当根发生变换的时候，只有两个点的轻重边发生了变化，其余轻重边并没有改变，所以我们只需要维护一个最大值和次大值就可以实现换根了；

时间复杂度 $O(n)$ ；

一个矩阵，两种操作一种操作可以选一个左下角矩阵覆盖，另一种可以选一个右上角矩阵覆盖，两种覆盖不能有交，求使用两种操作把矩阵覆盖满的方案数；

双向监督

子任务一：搜索所有的覆盖方案，判断是否合法

双向监督

子任务一：搜索所有的覆盖方案，判断是否合法

子任务二：注意到最后的方案一定是一个如下的折线；

双向监督

考虑动态规划 $dp_{i,j,k}$ 表示第 j 列的上界在第 i 行, 且前面有 k 个拐角;

双向监督

考虑动态规划 $dp_{i,j,k}$ 表示第 j 列的上界在第 i 行，且前面有 k 个拐角；转移为：

$$dp_{i,j,k} = dp_{i,j-1,k} + [c_{i-1,j} = 0] \sum_{l=1}^{i-1} dp_{l,j-1,k-2} \times [c_{l,j-1} = 0]$$

双向监督

考虑动态规划 $dp_{i,j,k}$ 表示第 j 列的上界在第 i 行，且前面有 k 个拐角；转移为：

$$dp_{i,j,k} = dp_{i,j-1,k} + [c_{i-1,j} = 0] \sum_{l=1}^{i-1} dp_{l,j-1,k-2} \times [c_{l,j-1} = 0]$$

注意到后段求和部分可以进行前缀和优化，所以复杂度是 $O(n^3)$ 的；

双向监督

考虑动态规划 $dp_{i,j,k}$ 表示第 j 列的上界在第 i 行，且前面有 k 个拐角；转移为：

$$dp_{i,j,k} = dp_{i,j-1,k} + [c_{i-1,j} = 0] \sum_{l=1}^{i-1} dp_{l,j-1,k-2} \times [c_{l,j-1} = 0]$$

注意到后段求和部分可以进行前缀和优化，所以复杂度是 $O(n^3)$ 的；此时答案为 $\sum_k dp_{n+1,n,k} \times 2^{cnt-k} + \sum_{i=1}^n dp_{i,n,k} \times 2^{cnt-k-1} \times [c_{i,n} = 0]$

双向监督

注意到 k , 对 $dp_{i,j,k}$ 最终计入答案的影响只是乘以 2^{cnt-k} (或 $2^{cnt-k-1}$, 根据 k 决定)

双向监督

注意到 k , 对 $dp_{i,j,k}$ 最终计入答案的影响只是乘以 2^{cnt-k} (或 $2^{cnt-k-1}$, 根据 k 决定)

令 $g_{i,j,k} = dp_{i,j,k} 2^{-k}$, $G_{i,j} = \sum g_{i,j,k}$;

双向监督

注意到 k , 对 $dp_{i,j,k}$ 最终计入答案的影响只是乘以 2^{cnt-k} (或 $2^{cnt-k-1}$, 根据 k 决定)

令 $g_{i,j,k} = dp_{i,j,k} 2^{-k}$, $G_{i,j} = \sum g_{i,j,k}$;
转移为:

$$G_{i,j} = G_{i,j-1} + [c_{i-1,j} = 0] \sum_{l=1}^{i-1} G_{l,j-1} / 4 \times [c_{l,j-1} = 0]$$

双向监督

注意到 k , 对 $dp_{i,j,k}$ 最终计入答案的影响只是乘以 2^{cnt-k} (或 $2^{cnt-k-1}$, 根据 k 决定)

令 $g_{i,j,k} = dp_{i,j,k} 2^{-k}$, $G_{i,j} = \sum g_{i,j,k}$;
转移为:

$$G_{i,j} = G_{i,j-1} + [c_{i-1,j} = 0] \sum_{l=1}^{i-1} G_{l,j-1} / 4 \times [c_{l,j-1} = 0]$$

复杂度为 $O(n^2)$;

Q & A